Лабораторна робота №10
Використання програми Postman для емуляції дій користувача сайту

Крайняк Михайло Юрійович, 451 група

1. Мета, середовище, підготовка

Мета: навчитися складати HTTP-запити в Postman, розрізняти параметри URL, заголовки й тіло, перевіряти відповіді та пояснювати причини помилок; дослідити локальний PHP API «Список завдань», виконати цикл створення, читання, оновлення й видалення запису та зібрати відтворювану колекцію перевірок.

Версії: PHP 8.2.12 (XAMPP; розширення mbstring, ctype, fileinfo). Настільний Postman на комп'ютері не встановлено, а Collection Runner у Postman потребує облікового запису, тому колекцію та середовище складено у форматі **Postman Collection v2.1** (імпортуються в Postman без змін) і виконано офіційним CLI-ранером Postman **Newman 6.2.3** (рушій PostmanRuntime 7.39.1, той самий, що в Postman; Node.js 22.23.1). Скрипти Pre-request і Post-response (`pm.test`, `pm.environment`) виконуються Newman так само, як у Postman.

Адреса запуску: `http://127.0.0.1:8080/api.php`. **Підготовка даних:** у каталозі `src` виконати `php reset.php` (відновлює 2 початкові записи), потім `php -d display_errors=0 -d log_errors=1 -S 127.0.0.1:8080`. Перед кожним повним прогоном — знову `php reset.php`.

```
Відновлено 2 початкові завдання. Сховище: C:\Users\teste\AppData\Local\Temp/php-lab10-6ed6a73f6397/tasks.json
```

2. Готовий і власний код

Файл	Походження	Зміст
`src/common.php`, `src/reset.php`, `src/sample.txt`	Готові, без змін	Валідація, JSON-відповіді, сховище з блокуванням, відновлення даних, файл для multipart
`src/api.php`	Готовий + власні рядки	Обробники tasks, echo, auth; додано підключення `search.php` і фільтр q (рядки з коментарем «Власне доповнення»)
`src/search.php`	Власне	`searchQuery()`, `titleMatches()` — перевірка й застосування q
`postman/lab10.postman_collection.json`, `postman/lab10.postman_environment.json`	Власне	Колекція (41 запит, 5 папок, тести в кожному) і середовище `Lab10 Local`

Файл	Походження	Зміст
`results/*`	Результати	Вивід і JSON-журнал Newman, HTML-звіт, самоперевірка тесту, прогін колекції викладача
`tools/build_collection.py`, `tools/render_evidence.py`	Власне	Генерація колекції; сторінки «запит — відповідь — Test Results» для знімків із журналу Newman

3. Колекція та середовище (кроки 2, 11, 12)

Колекція **«ЛР 10 — HTTP і Postman (Крайняк М. Ю., 451)»**, Authorization на рівні колекції — **No Auth**. Папки за порядком сценарію:

1. **01 — Основний сценарій** (15): список → фільтри done=1/0 → один запис → id+done (400) → urlencoded-форма → створення (201) → читання → PUT → PUT без done (422) → підтвердження PUT → DELETE (204) → 404 → повторний DELETE (404) → список.
2. **02 — Помилкові запити** (12): таблиця кроку 9 і додаткові 422 (рядок "false", поле id у тілі), наприкінці перевірка, що невдалі POST нічого не створили.
3. **03 — Basic Auth** (3): No Auth → 401, правильні демонстраційні дані → 200, неправильний пароль → 401.
4. **04 — Пошук q (власне доповнення)** (10).
5. **05 — Ручне завантаження файла** (1): multipart з `attachment`; у Postman файл обирають вручну, Newman бере `sample.txt` з `--working-dir src`.

Середовище «Lab10 Local»: `base_url = http://127.0.0.1:8080/api.php` (без query), `task_id` — порожній. У всіх запитах адреса `{{base_url}}`, параметри — у Params (query). Запит «04 — Створити завдання» у **Pre-request** виконує `pm.environment.unset("task_id")`, а в **Post-response** після 201 зберігає новий id: `pm.environment.set("task_id", body.data.id)`; подальші запити використовують `{{task_id}}`, а не «3». Кожен запит має тести і статусу, і змісту: JSON-тип, структура `data/meta/error`, конкретні значення, заголовки Location, Allow, WWW-Authenticate, порожнє тіло 204.

```
// Post-response «04 — Створити завдання» (фрагмент колекції)
pm.test("Статус 201 Created", () => pm.response.to.have.status(201));
pm.test("Новий запис", () => { pm.expect(body.data.id).to.be.a("number").above(0);
    pm.expect(body.data.title).to.equal("Перевірити POST"); pm.expect(body.data.done).to.equal(false); });
pm.test("Location веде на новий запис", () =>
    pm.expect(pm.response.headers.get("Location")).to.equal("api.php?resource=tasks&id=" + body.data.id));
if (pm.response.code === 201 && Number.isInteger(body.data.id)) { pm.environment.set("task_id",
body.data.id); }

// Post-response «08 — Видалити завдання»
pm.test("Статус 204 No Content", () => pm.response.to.have.status(204));
pm.test("Тіло порожнє", () => pm.expect(pm.response.text()).to.equal(""));
```

4. Query, urlencoded, multipart і raw JSON

Спосіб	Мій запит	Де дані і що бачить сервер
Query-параметри	«02 — Фільтр виконаних»: `?resource=tasks&done=1`	У URL; завжди рядки (`"1"`, а не `true`); PHP — `$_GET`. Масив `id[]=1` → 400.
x-www-form-urlencoded	«03 — URL-encoded форма»: тіло `name=Студент DEMO&group=DEMO`	У тілі, пари ключ=значення з URL-кодуванням; PHP — `$_POST`. `resource=echo` лишається в URL і в `fields` не потрапляє.
multipart/form-data	«Вручну — Вкладення attachment»: поле name + файл sample.txt	Тіло з частинами, розділеними boundary (Content-Type з boundary ставить клієнт); PHP — `$_POST` і `$_FILES`; сервер повернув name, size = 150, detected_mime text/plain.
raw JSON	«04 — Створити завдання»: `{"title": "Перевірити POST", "done": false}`	Тіло з `Content-Type: application/json`; `$_POST` порожній, тіло читається з `php://input` і `json_decode`; типи JSON зберігаються (`false` — логічне).

5. Знімки ключових етапів

Знімки побудовано з журналу Newman (`results/newman-run.json`): для кожного запиту показано метод і URL, заголовки й тіло запиту, статус, заголовки й тіло відповіді та Test Results — ті самі панелі, що в Postman. Службові заголовки клієнта (User-Agent, Postman-Token тощо) приховано, значення Authorization закрито.

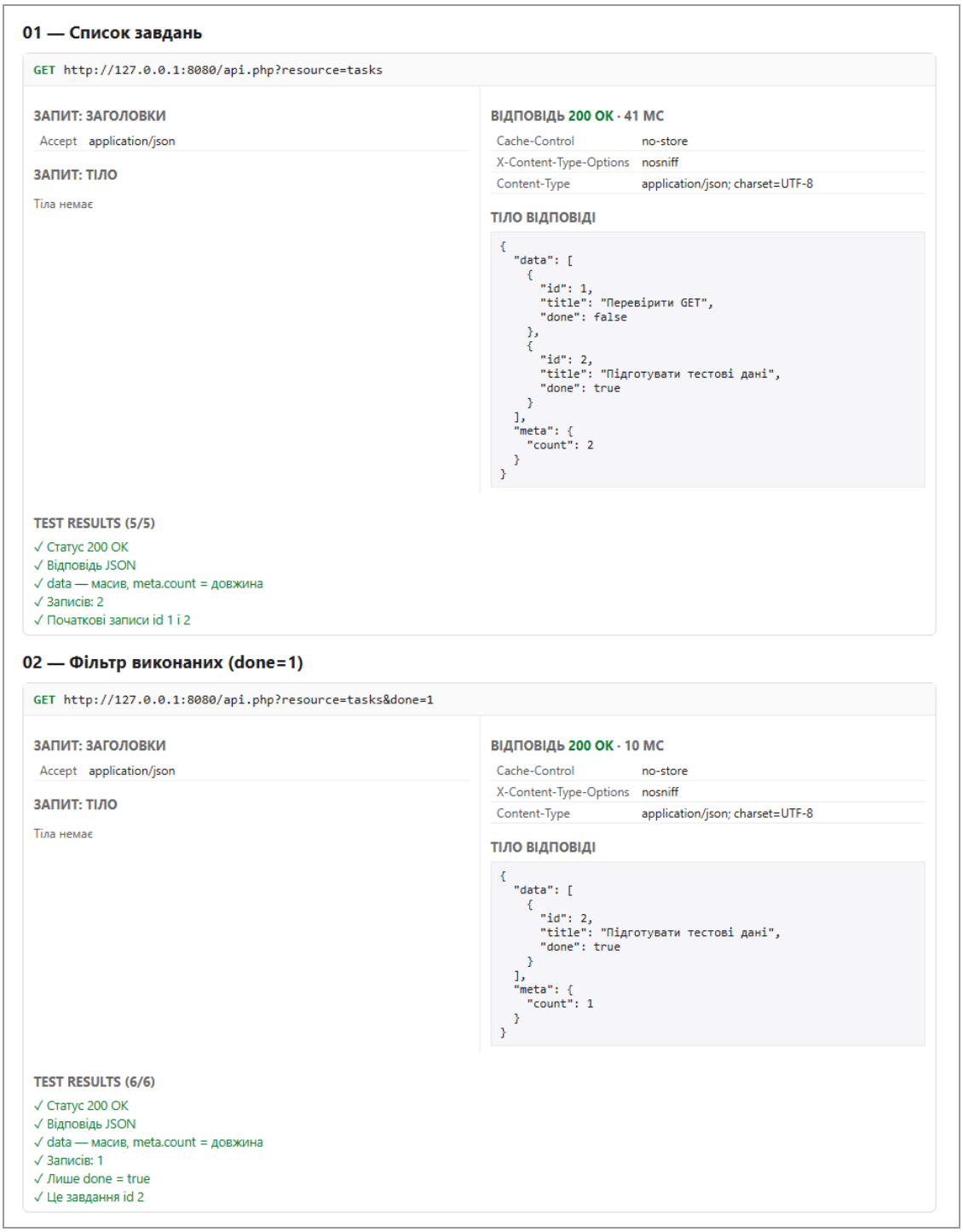


Рис. 1. GET списку: 200, `Content-Type: application/json; charset=UTF-8`, 2 записи, `meta.count = 2`; фільтр `done=1` — один виконаний запис.

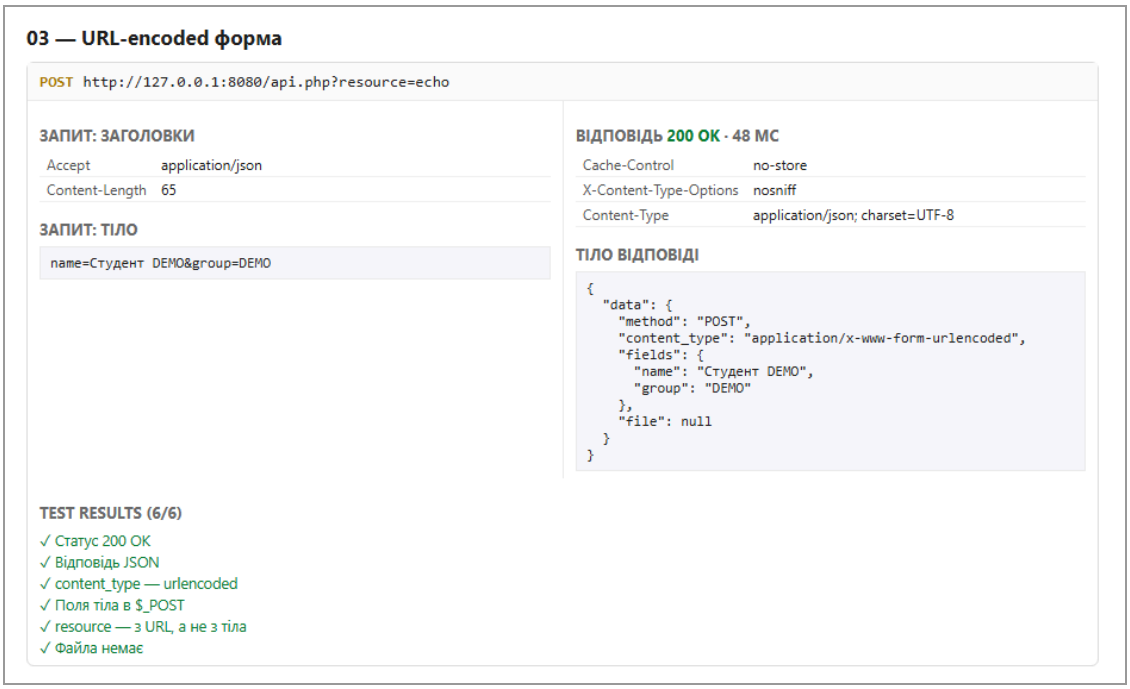


Рис. 2. POST `resource=echo` з тілом x-www-form-urlencoded: поля потрапили в `$_POST`.

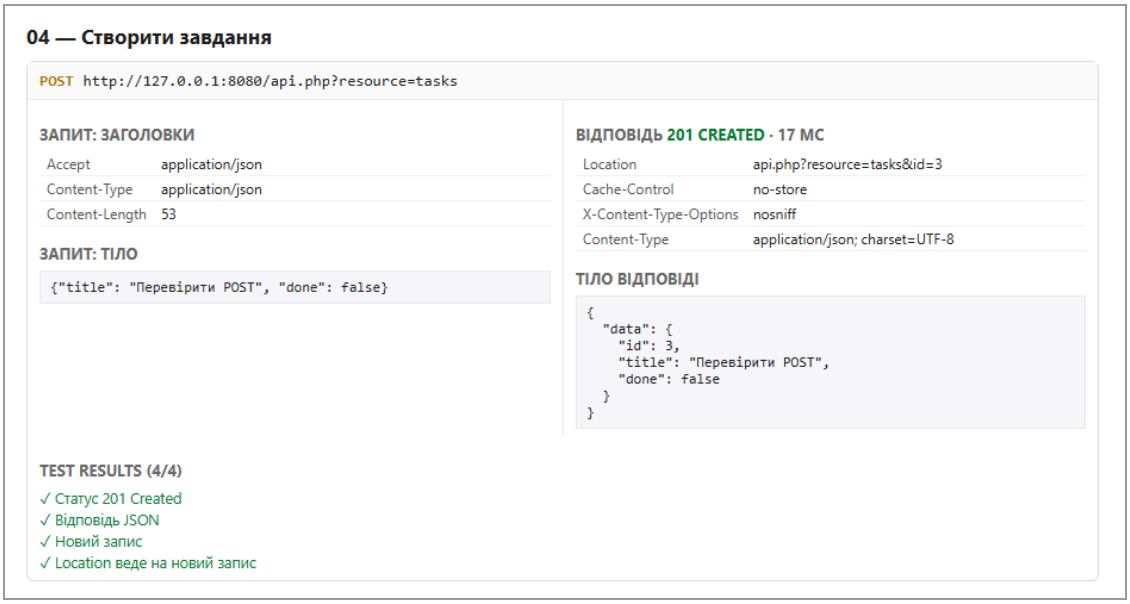


Рис. 3. POST із raw JSON: 201 Created, новий id = 3, заголовок `Location: api.php?resource=tasks&id=3`; id збережено в `task_id`.

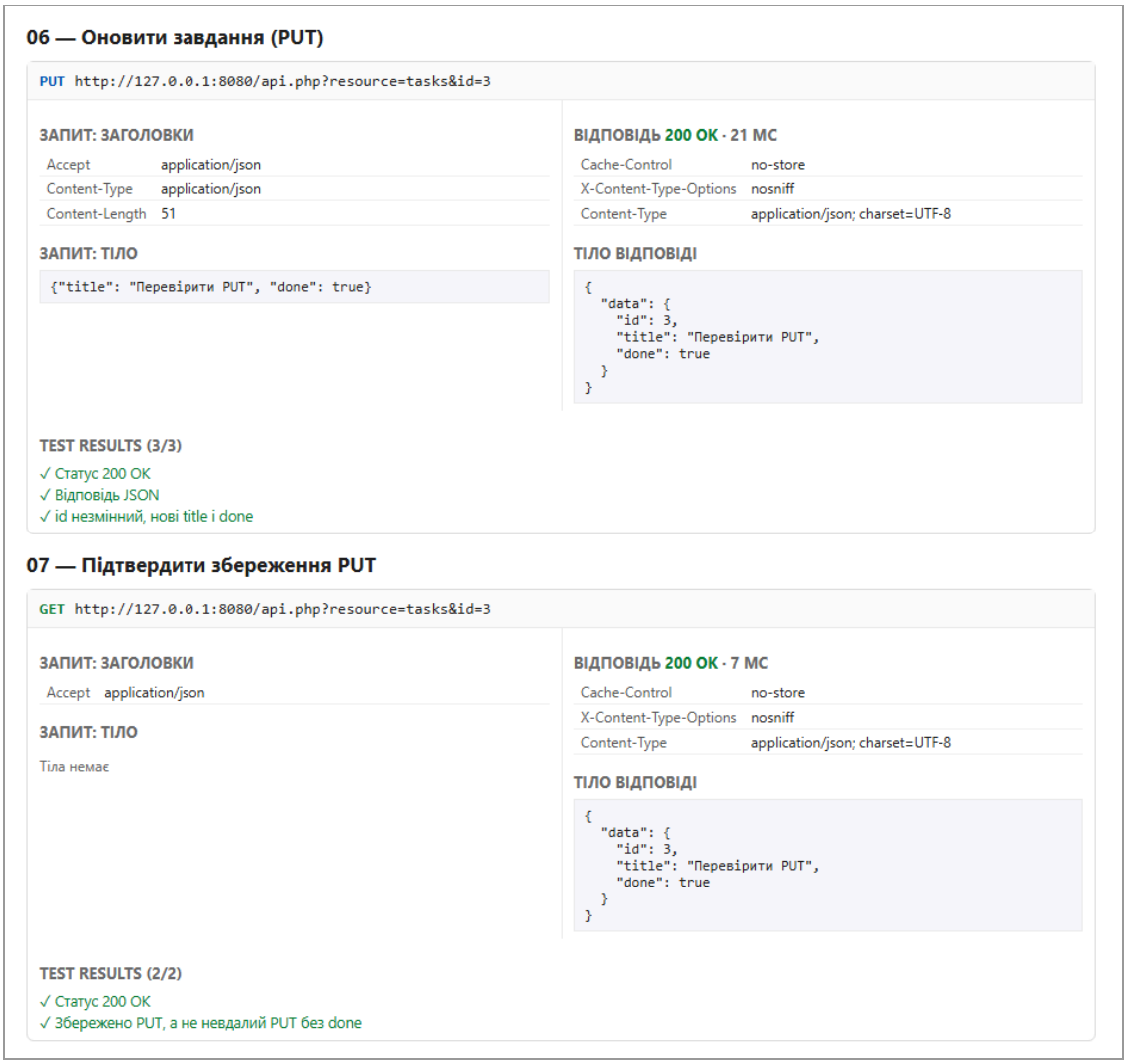


Рис. 4. PUT за `{{task_id}}`: 200, той самий id, новий title і done = true; наступний GET підтверджує збереження.

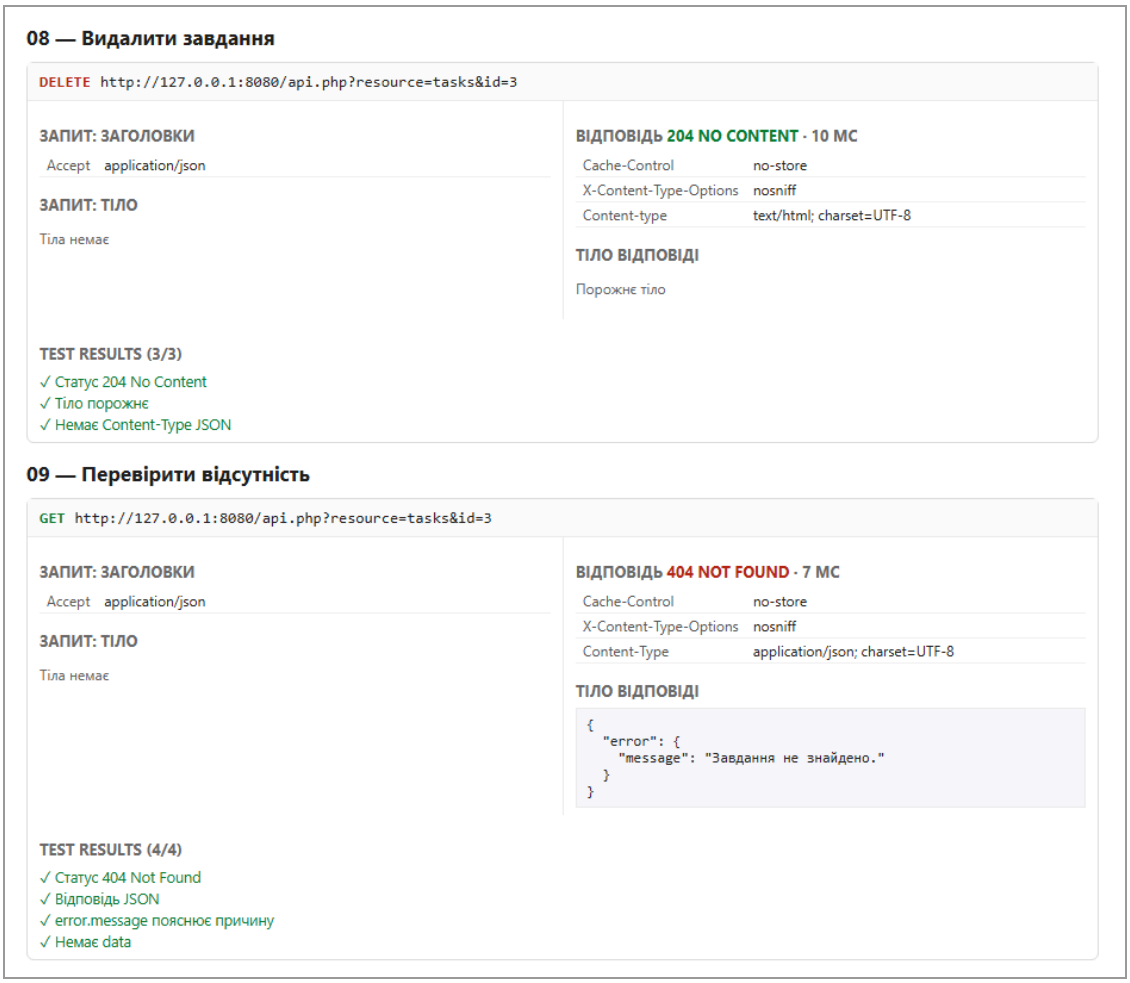


Рис. 5. DELETE: 204 No Content із порожнім тілом; GET того самого id: 404 з `error.message`.

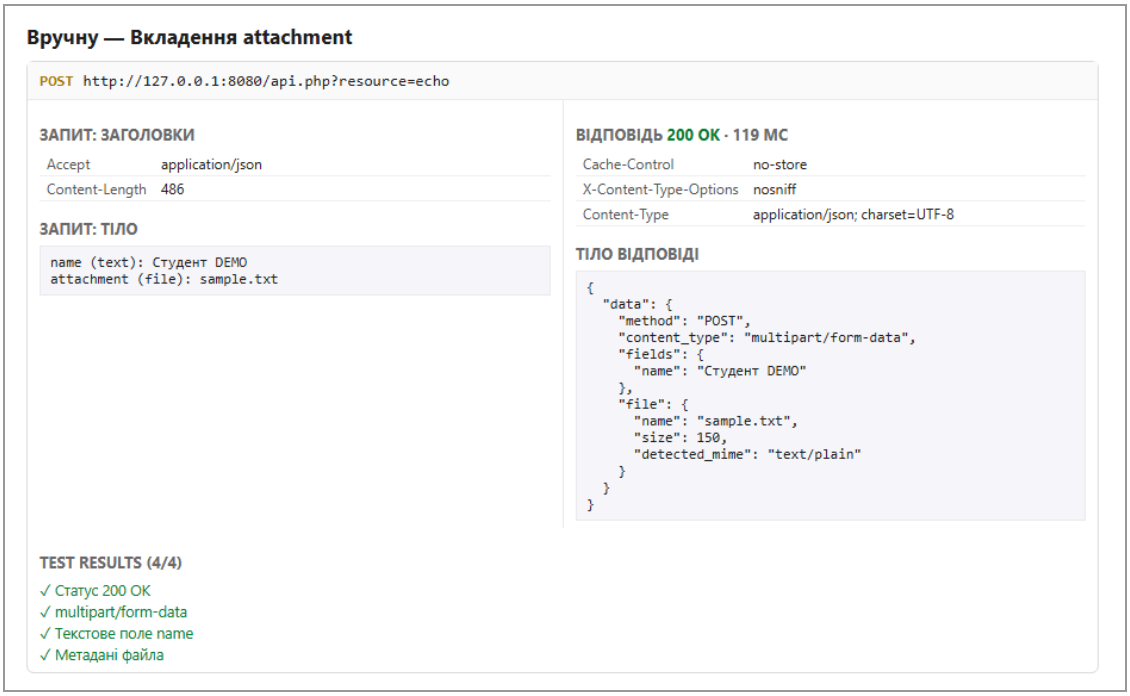


Рис. 6. Вкладення: multipart/form-data з boundary; сервер повернув name, size = 150 байтів (як у файла) і detected_mime = text/plain.

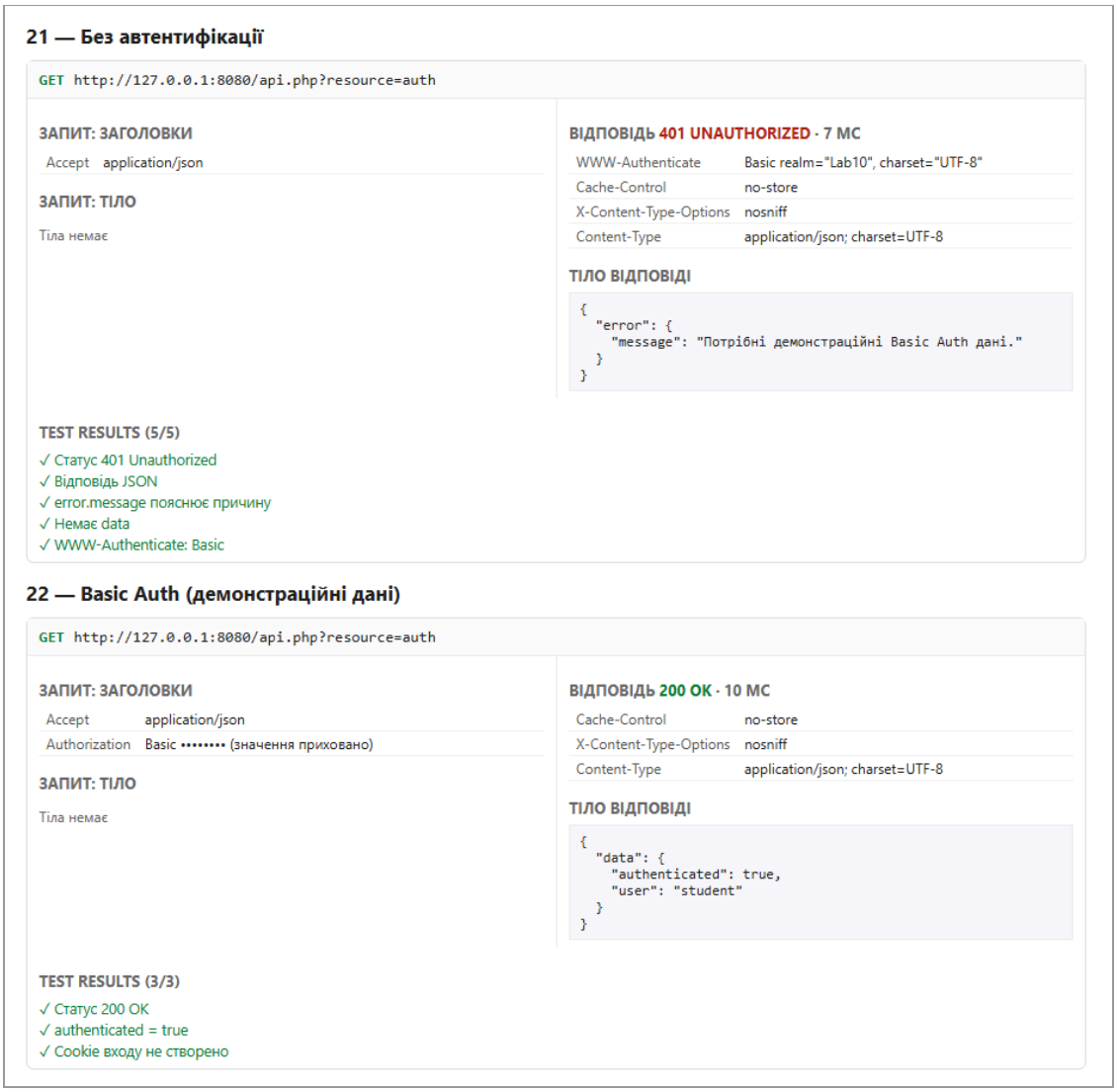


Рис. 7. Basic Auth: без автентифікації — 401 і `WWW-Authenticate: Basic realm="Lab10"`; з демонстраційними даними — 200, `authenticated = true`, cookie не створюється. Значення Authorization закрито.

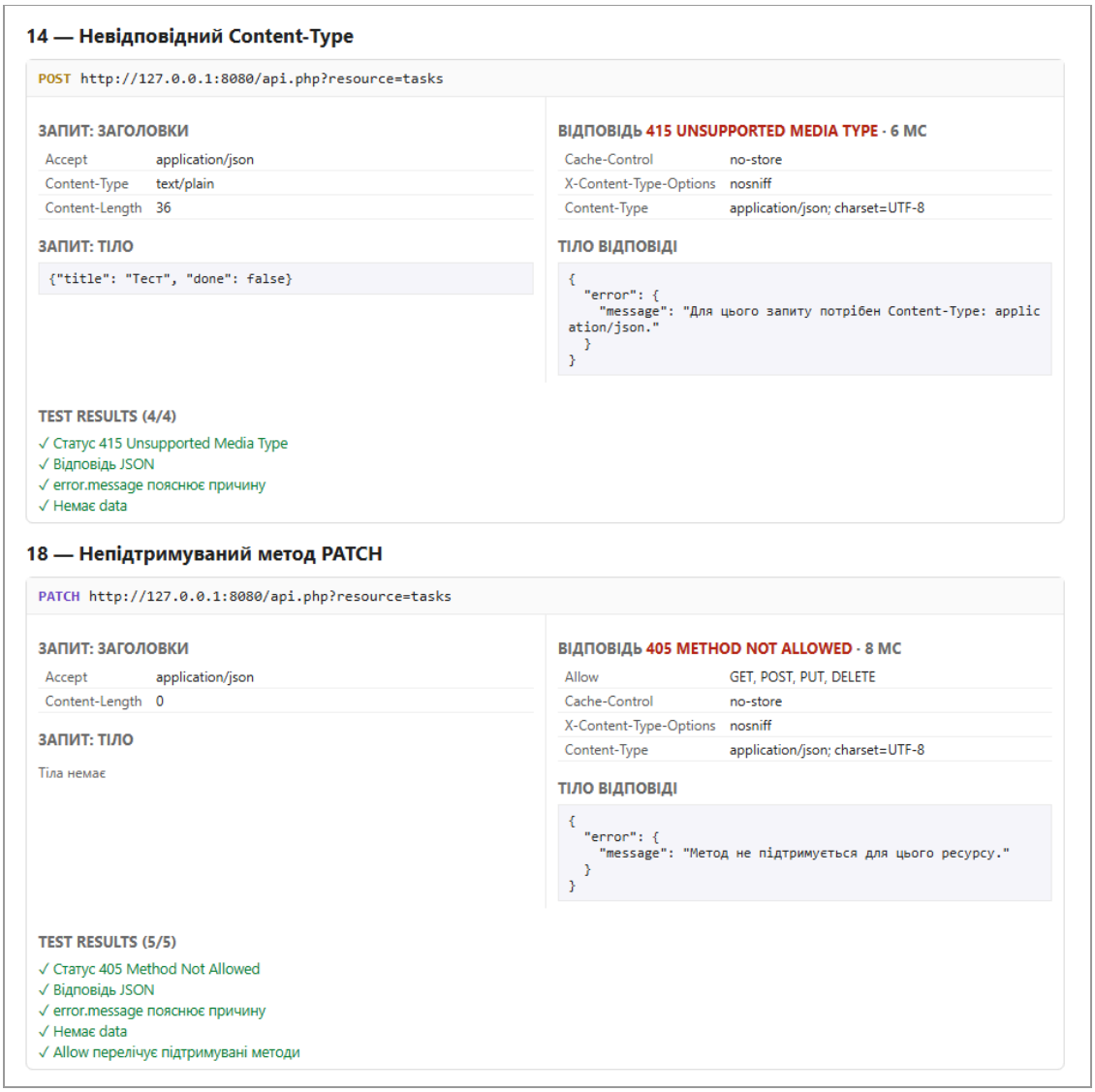


Рис. 8. Помилкові запити: text/plain → 415; PATCH → 405 і `Allow: GET, POST, PUT, DELETE`.

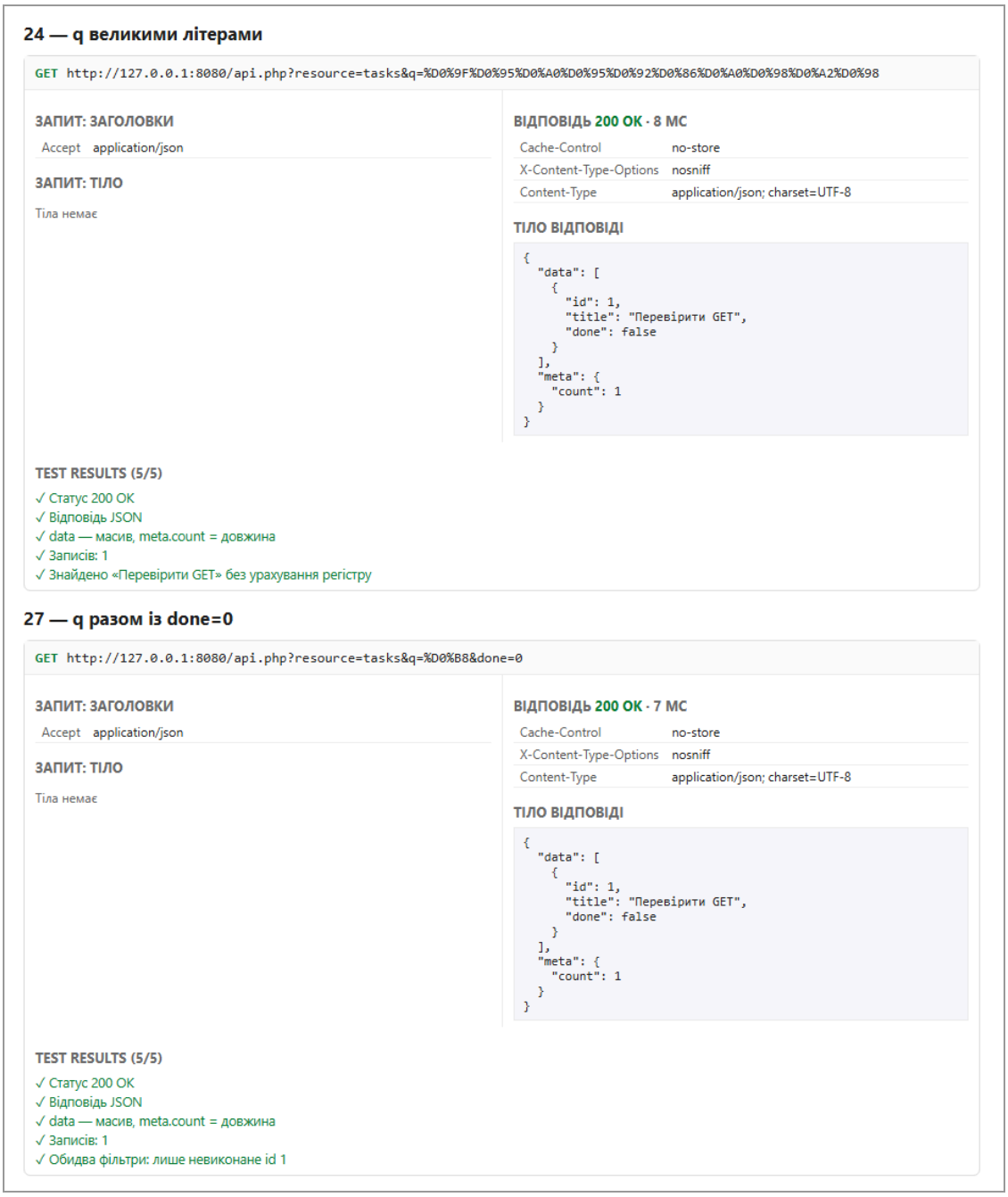


Рис. 9. Власний фільтр q: «ПЕРЕВІРИТИ» знаходить «Перевірити GET»; `q=и&done=0` застосовує обидва фільтри.

Collection Runner (Newman): ЛР 10 — HTTP і Postman · середовище Lab10 Local · 1 ітерація

Запитів: 41 (помилок 0) · перевірок: 172 (невдалих 0) · тестових скриптів: 41

Папка	Запит	Метод	Статус	Тести
01	01 — Список завдань	GET	200 OK	5/5
01	02 — Фільтр виконаних (done=1)	GET	200 OK	6/6
01	02a — Фільтр невиконаних (done=0)	GET	200 OK	5/5
01	02b — Один запис (id=1)	GET	200 OK	4/4
01	02c — id разом із done	GET	400 Bad Request	4/4
01	03 — URL-encoded форма	POST	200 OK	6/6
01	04 — Створити завдання	POST	201 Created	4/4
01	05 — Прочитати створене	GET	200 OK	2/2
01	06 — Оновити завдання (PUT)	PUT	200 OK	3/3
01	06a — PUT без done	PUT	422 Unknown Status Code	4/4
01	07 — Підтвердити збереження PUT	GET	200 OK	2/2
01	08 — Видалити завдання	DELETE	204 No Content	3/3
01	09 — Перевірити відсутність	GET	404 Not Found	4/4
01	09a — Повторний DELETE	DELETE	404 Not Found	4/4
01	10 — Список після циклу CRUD	GET	200 OK	5/5
02	11 — Порожній title	POST	422 Unknown Status Code	4/4
02	12 — Неправильні типи	POST	422 Unknown Status Code	4/4
02	12a — Рядок "false" замість false	POST	422 Unknown Status Code	4/4
02	12b — Поле id у тілі	POST	422 Unknown Status Code	4/4
02	13 — Пошкоджений JSON	POST	400 Bad Request	4/4
02	14 — Невідповідний Content-Type	POST	415 Unsupported Media Type	4/4
02	15 — Масив у query (id[])	GET	400 Bad Request	4/4
02	16 — Невідомий запис	GET	404 Not Found	4/4
02	17 — Невідомий ресурс	GET	404 Not Found	4/4
02	18 — Непідтримуваний метод PATCH	PATCH	405 Method Not Allowed	5/5
02	19 — DELETE без id	DELETE	400 Bad Request	4/4
02	20 — Невдалі POST не створили записів	GET	200 OK	5/5
03	21 — Без автентифікації	GET	401 Unauthorized	5/5
03	22 — Basic Auth (демонстраційні дані)	GET	200 OK	3/3
03	23 — Basic Auth з неправильним паролем	GET	401 Unauthorized	4/4
04	24 — q великими літерами	GET	200 OK	5/5
04	25 — q іншим регістром у середині слова	GET	200 OK	5/5
04	26 — q без збігів	GET	200 OK	5/5
04	27 — q разом із done=0	GET	200 OK	5/5
04	27a — Той самий q без done	GET	200 OK	4/4
04	28 — Масив q[]	GET	400 Bad Request	4/4
04	29 — q разом з id	GET	400 Bad Request	4/4
04	30 — q довший за 40 символів	GET	400 Bad Request	4/4
04	31 — q лише з пробілів = без пошуку	GET	200 OK	4/4
04	32 — q з пробілами по краях (trim)	GET	200 OK	5/5
05	Вручну — Вкладення attachment	POST	200 OK	4/4

Рис. 10. Підсумок прогону колекції (аналог Collection Runner): 41 запит, 172 перевірки, 0 невдалих, одна ітерація.

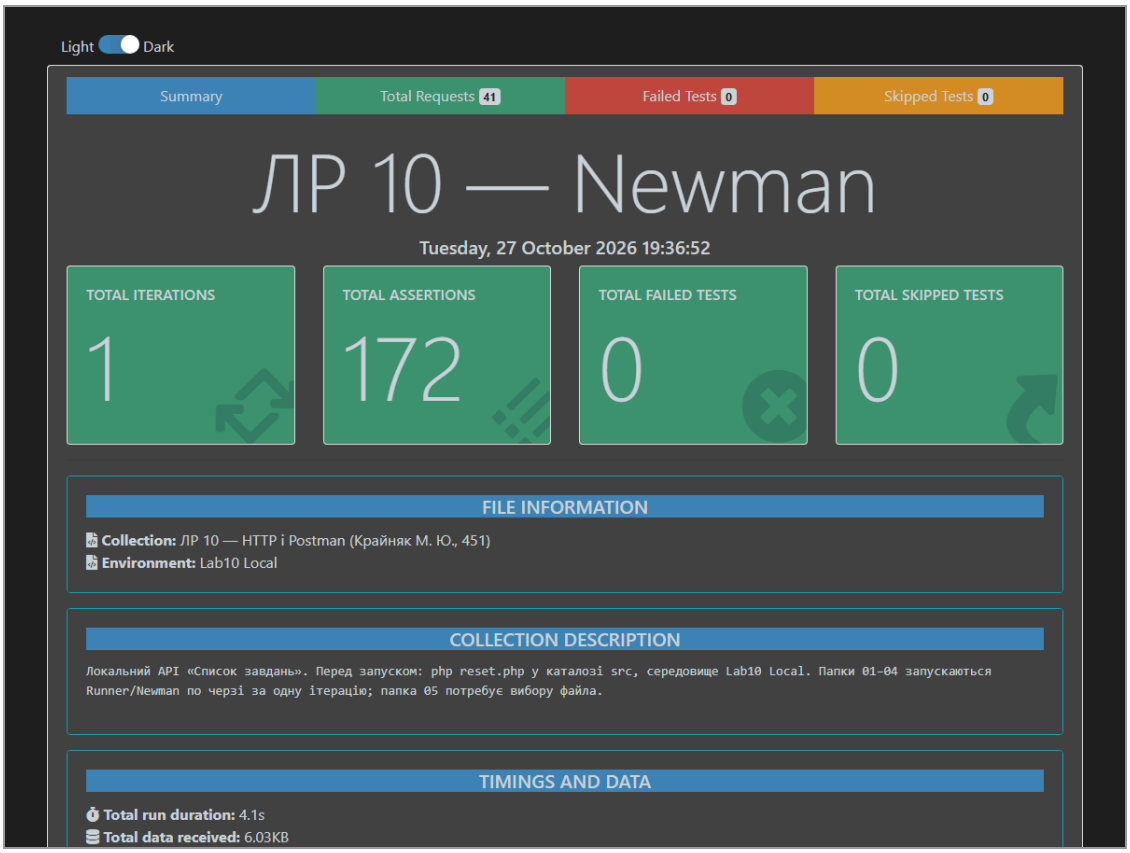


Рис. 11. HTML-звіт Newman (htmlextra, `results/newman-report.html`): 1 ітерація, 172 перевірки, 0 невдалих.

6. Таблиця запитів і фактичних результатів

Для GET і DELETE тіла немає («немає»). Усі запити мають заголовок `Accept: application/json`. Фактичний результат — з прогону Newman після `php reset.php`.

Запит	Метод	Query	Заголовки (крім Accept)	Тіло	Очікування	Фактично	Висновок
01 — Список завдань	GET	`resource=tasks`	—	немає	200; 2 записи	200; тести 5/5	відповідає
02 — Фільтр виконаних (done=1)	GET	`resource=tasks&done=1`	—	немає	200; 1 запис done=true	200; тести 6/6	відповідає
02a — Фільтр невиконаних (done=0)	GET	`resource=tasks&done=0`	—	немає	200; 1 запис done=false	200; тести 5/5	відповідає
02b — Один запис (id=1)	GET	`resource=tasks&id=1`	—	немає	200; об’єкт id 1	200; тести 4/4	відповідає
02c — id разом із done	GET	`resource=tasks&id=1&done=1`	—	немає	400	400; тести 4/4	відповідає
03 — URL-encoded форма	POST	`resource=echo`	Content-Type: application/x-www-form-urlencoded (автоматично)	`urlencoded: name=Студент DEMO&group=DEMO`	200; поля name, group	200; тести 6/6	відповідає
04 — Створити завдання	POST	`resource=tasks`	Content-Type: application/json	`{"title": "Перевірити POST", "done": false}`	201; новий id, Location	201; тести 4/4	відповідає
05 — Прочитати створене	GET	`resource=tasks&id={{task_id}}`	—	немає	200; той самий запис	200; тести 2/2	відповідає

Запит	Метод	Query	Заголовки (крім Accept)	Тіло	Очікування	Фактично	Висновок
06 — Оновити завдання (PUT)	PUT	`resource=tasks&id={{task_id}}`	Content-Type: application/json	`{"title": "Перевірити PUT", "done": true}`	200; title/done змінено	200; тести 3/3	відповідає
06a — PUT без done	PUT	`resource=tasks&id={{task_id}}`	Content-Type: application/json	`{"title": "Без done"}`	422; без змін	422; тести 4/4	відповідає
07 — Підтвердити збереження PUT	GET	`resource=tasks&id={{task_id}}`	—	немає	200; зміни PUT збережено	200; тести 2/2	відповідає
08 — Видалити завдання	DELETE	`resource=tasks&id={{task_id}}`	—	немає	204; запис видалено	204; тести 3/3	відповідає
09 — Перевірити відсутність	GET	`resource=tasks&id={{task_id}}`	—	немає	404	404; тести 4/4	відповідає
09a — Повторний DELETE	DELETE	`resource=tasks&id={{task_id}}`	—	немає	404; без змін	404; тести 4/4	відповідає
10 — Список після циклу CRUD	GET	`resource=tasks`	—	немає	200; знову 2 записи	200; тести 5/5	відповідає
11 — Порожній title	POST	`resource=tasks`	Content-Type: application/json	`{"title": " ", "done": false}`	422; запис не створено	422; тести 4/4	відповідає
12 — Неправильні типи	POST	`resource=tasks`	Content-Type: application/json	`{"title": [], "done": "false"}`	422; запис не створено	422; тести 4/4	відповідає
12a — Рядок "false" замість false	POST	`resource=tasks`	Content-Type: application/json	`{"title": "Тест", "done": "false"}`	422	422; тести 4/4	відповідає
12b — Поле id у тілі	POST	`resource=tasks`	Content-Type: application/json	`{"id": 77, "title": "Тест", "done": false}`	422	422; тести 4/4	відповідає
13 — Пошкоджений JSON	POST	`resource=tasks`	Content-Type: application/json	`{"title": "Тест",}`	400	400; тести 4/4	відповідає
14 — Невідповідний Content-Type	POST	`resource=tasks`	Content-Type: text/plain	`{"title": "Тест", "done": false}`	415	415; тести 4/4	відповідає
15 — Масив у query (id[])	GET	`resource=tasks&id[]=1`	—	немає	400	400; тести 4/4	відповідає
16 — Невідомий запис	GET	`resource=tasks&id=999999999`	—	немає	404	404; тести 4/4	відповідає
17 — Невідомий ресурс	GET	`resource=unknown`	—	немає	404	404; тести 4/4	відповідає
18 — Непідтримуваний метод PATCH	PATCH	`resource=tasks`	—	немає	405; Allow	405; тести 5/5	відповідає
19 — DELETE без id	DELETE	`resource=tasks`	—	немає	400	400; тести 4/4	відповідає
20 — Невдалі POST не створили записів	GET	`resource=tasks`	—	немає	200; лише id 1 і 2	200; тести 5/5	відповідає
21 — Без автентифікації	GET	`resource=auth`	—	немає	401; WWW-Authenticate	401; тести 5/5	відповідає
22 — Basic Auth (демонстраційні дані)	GET	`resource=auth`	Authorization: Basic (Postman)	немає	200; authenticated	200; тести 3/3	відповідає
23 — Basic Auth з неправильним паролем	GET	`resource=auth`	Authorization: Basic (Postman)	немає	401	401; тести 4/4	відповідає
24 — q великими літерами	GET	`resource=tasks&q=ПЕРЕВІРИТИ`	—	немає	200; 1 збіг	200; тести 5/5	відповідає
25 — q іншим регістром у середині слова	GET	`resource=tasks&q=ТестОві`	—	немає	200; 1 збіг	200; тести 5/5	відповідає
26 — q без збігів	GET	`resource=tasks&q=автобус`	—	немає	200; 0 записів	200; тести 5/5	відповідає

Запит	Метод	Query	Заголовки (крім Accept)	Тіло	Очікування	Фактично	Висновок
27 — q разом із done=0	GET	`resource=tasks&q=и&done=0`	—	немає	200; 1 (q і done)	200; тести 5/5	відповідає
27a — Той самий q без done	GET	`resource=tasks&q=и`	—	немає	200; 2	200; тести 4/4	відповідає
28 — Масив q[]	GET	`resource=tasks&q[]=Перевірити`	—	немає	400	400; тести 4/4	відповідає
29 — q разом з id	GET	`resource=tasks&id=1&q=Перевірити`	—	немає	400	400; тести 4/4	відповідає
30 — q довший за 40 символів	GET	`resource=tasks&q=ааааааааааааааааааааааааааааааааааааааа`	—	немає	400	400; тести 4/4	відповідає
31 — q лише з пробілів = без пошуку	GET	`resource=tasks&q=`	—	немає	200; 2 (без пошуку)	200; тести 4/4	відповідає
32 — q з пробілами по краях (trim)	GET	`resource=tasks&q= дані`	—	немає	200; 1 (trim)	200; тести 5/5	відповідає
Вручну — Вкладення attachment	POST	`resource=echo`	Content-Type: multipart/form-data; boundary=… (автоматично)	`form-data: name=Студент DEMO, attachment=файл sample.txt`	200; метадані файла	200; тести 4/4	відповідає

Примітка. Вбудований сервер PHP 8.2 не має текстової назви для коду 422 і надсилає рядок стану «422 Unknown Status Code». Код правильний; Apache надіслав би «Unprocessable Entity». Тести перевіряють саме код.

7. Власне доповнення: фільтр q

Правила: q — рядок (масив `q[]` → 400), після `trim()` не довший за 40 символів (інакше 400), порожній q — пошуку немає; збіг підрядка в title без урахування регістру (`mb_strtolower`, тож працює й для української); q разом із done застосовує обидва фільтри; q разом з id → 400, як і done з id. Перевірка виконується до звернення до сховища.

```
<?php
declare(strict_types=1);

// Власне доповнення (завдання для самостійного виконання): фільтр q у GET списку завдань.

const SEARCH_MAX_LENGTH = 40;

/**
 * Пошуковий рядок із query-параметра q.
 * null — пошуку немає (параметр відсутній або порожній після trim()).
 */
function searchQuery(): ?string
{
    $raw = queryString('q'); // Масив q[] → 400 «Параметр q має бути рядком».
    if ($raw === null) {
        return null;
    }
    if (!mb_check_encoding($raw, 'UTF-8')) {
        throw new ApiError(400, 'q має бути рядком UTF-8.');
    }
    $query = trim($raw);
    if ($query === '') {
        return null;
    }
```

```
    if (mb_strlen($query, 'UTF-8') > SEARCH_MAX_LENGTH) {
        throw new ApiError(400, 'q після trim() має містити не більше ' . SEARCH_MAX_LENGTH . '
символів.');
    }
    return $query;
}

/** Чи містить title підрядок q без урахування регістру (українські літери теж). */
function titleMatches(string $title, string $query): bool
{
    return str_contains(mb_strtolower($title, 'UTF-8'), mb_strtolower($query, 'UTF-8'));
}
```

Зміни в готовому `api.php` (diff):

```
@@ require
 require __DIR__ . '/common.php';
+require __DIR__ . '/search.php'; // Власне доповнення: фільтр q.
@@ GET
         if ($id !== null && $done !== null) {
             throw new ApiError(400, 'Для одного запису не додавайте фільтр done.');
         }
-        $result = withStore(function (array &$store) use ($id, $done): array {
+        // Власне доповнення: q перевіряється до звернення до сховища; q з id відхиляємо, як і done.
+        $query = searchQuery();
+        if ($id !== null && $query !== null) {
+            throw new ApiError(400, 'Для одного запису не додавайте фільтр q.');
+        }
+        $result = withStore(function (array &$store) use ($id, $done, $query): array {
@@ фільтрація
             if ($done !== null) { ... }
+            if ($query !== null) { // Власне доповнення: обидва фільтри застосовуються разом.
+                $tasks = array_values(array_filter($tasks, fn(array $task): bool =>
titleMatches($task['title'], $query)));
+            }
```

Перевірки q — запити 24–32 (рис. 9 і таблиця): інший регістр української (ПЕРЕВІРИТИ, ТестОві), відсутній збіг (порожній масив, не помилка), q з done, масив `q[]`, q з id, 41 символ, лише пробіли, пробіли по краях.

8. Перевірка самих тестів і сумісності

Самоперевірка тесту (крок 11). У копії запиту «01 — Список завдань» очікуваний статус тимчасово замінено на 201. Тест став невдалим, інші перевірки запиту пройшли — отже, тест справді перевіряє статус. У робочій колекції очікування правильне (200).

```
└ 01 — Список завдань
  GET http://127.0.0.1:8080/api.php?resource=tasks [200 OK, 545B, 68ms]
  1. Статус 201 (навмисно неправильне очікування)
  √  Відповідь JSON
  √  data — масив, meta.count = довжина
  √  Записів: 2
  √  Початкові записи id 1 і 2
...
 1.  AssertionError  Статус 201 (навмисно неправильне очікування)
     expected response to have status code 201 but got 200
```

Сумісність. Готова колекція викладача (папка «01 — Основний сценарій», 15 запитів) після `reset.php` пройшла проти доповненого API: 15 запитів, 40 перевірок, 0 невдалих

(`results/newman-given-collection.txt`) — фільтр q не змінив наявного контракту.

9. Виявлені проблеми та виправлення

- Порт 8080 спершу був зайнятий phpMyAdmin із попередньої роботи — сервер зупинено, API запущено на адресі з методички.
- Залежні запити: 05–10 використовують `{{task_id}}`; щоб збій створення не підставив старий id, у Pre-request запиту 04 значення очищується. Перевірка «Записів: 2» вимагає `reset.php` перед прогоном — повторний прогін без нього дав би інший стан, тому підготовку даних включено в інструкцію.
- Для 204 тест не викликає `pm.response.json()` (тіла немає), а перевіряє `pm.response.text() === ""`.
- Для multipart у Newman файл задається шляхом відносно `--working-dir`; у Postman його обирають вручну (папка 05).
- Рядок стану «422 Unknown Status Code» вбудованого сервера PHP — особливість сервера, а не API; тести перевіряють код.
- Помилок у роботі API чи власного фільтра прогін не виявив: 41/41 запит і 172/172 перевірки з першого повного прогону.

10. Відповіді на контрольні питання

1. Метод, URL (з `{{base_url}}`), query-параметри (Params), заголовки (Headers), тіло (Body: none, form-data, x-www-form-urlencoded, raw, binary), авторизацію (Authorization), а також скрипти Pre-request і Post-response.
2. Query-параметри — частина URL після `?`, завжди рядки, PHP читає їх із `$_GET`, вони видні в журналах і закладках. Тіло передається після заголовків, має власний формат (Content-Type), може бути великим і структурованим (JSON, файли).
3. urlencoded — пари `ключ=значення` з %-кодуванням, лише текст; multipart/form-data — частини з окремими заголовками, розділені boundary, підходить для файлів; application/json — довільна структура з типами JSON (числа, логічні, масиви, об’єкти), читається з `php://input`.
4. `Content-Type` — формат тіла (запиту чи відповіді); `Accept` — бажаний клієнтом формат відповіді; `Location` — адреса створеного ресурсу (після 201); `Allow` — перелік дозволених методів (у відповіді 405).
5. PHP заповнює `$_POST` лише для `application/x-www-form-urlencoded` і `multipart/form-data`. JSON-тіло потрібно прочитати з `php://input` і розібрати `json_decode`.
6. 201 — запис створено; 204 — успіх без тіла (видалення); 400 — запит синтаксично неправильний (зіпсований JSON, неправильний query); 415 — непідтримуваний формат тіла (Content-Type); 422 — синтаксис правильний, але дані порушують контракт (порожній title, "false" замість false).

7. id генерує сервер; він залежить від попередніх операцій (після reset — 3, після інших створень — інший, видалені id не повторюються). Тому id беруть із відповіді 201 і зберігають у змінній `task_id`.
8. Після PUT виконати окремий GET того самого id і порівняти title/done (відповідь PUT лише повідомляє результат); після DELETE — GET того самого id має дати 404, а список — не містити id, при цьому інші записи лишаються.
9. Колекція — збережений набір запитів зі скриптами й папками; середовище — набір змінних для конкретного запуску (адреса, id); змінна — іменоване значення `{{base_url}}`, яке підставляється перед надсиланням з урахуванням області видимості (глобальна, колекції, середовища, локальна).
10. Pre-request — перед надсиланням запиту (підготовка змінних, очищення task_id); Post-response (Tests) — після отримання відповіді, на клієнті, а не на сервері.
11. Basic Auth передає «користувач:пароль» у Base64 — це кодування, яке будь-хто декодує, тому без HTTPS облікові дані перехоплюються. Заголовок Authorization надсилається з кожним запитом окремо; сервер не видає cookie й не запам'ятовує вхід — сесії не виникає.
12. Результати залежать від стану даних (кількість записів, наступний id). Без відомого початкового стану (`reset.php`) тести «2 записи» чи «id 1 і 2» хибно впадуть або хибно пройдуть.
13. Збій підключення (ECONNREFUSED, «Could not send request») означає, що HTTP-відповіді немає взагалі — сервер не запущено або неправильні хост/порт. HTTP-помилка (4xx/5xx) — сервер відповів статусом і тілом.
14. Правильний статус не гарантує правильного змісту: 200 може містити не ті записи, 201 — неправильні поля, а невдалий запит може все одно змінити дані. Тому перевіряють тип відповіді, структуру, значення, заголовки й стан даних після операції.

11. Висновок

Складено власну колекцію Postman із 41 запиту та середовище з `base_url` і `task_id`; на локальному API виконано повний цикл POST → GET → PUT → GET → DELETE → 404, досліджено query, urlencoded, multipart і raw JSON, помилкові запити (400, 404, 405, 415, 422) та Basic Auth. Кожен запит перевіряється тестами статусу й змісту, id створеного запису передається між запитами через змінну середовища. API доповнено фільтром q з валідацією та поєднанням із done. Прогін колекції (Newman, рушій Postman) після reset.php — 172 перевірки без помилок, неправильні запити записів не створюють; навмисно зіпсований тест падає, а готова колекція викладача сумісна з доповненим API.

12. Посилання на репозиторій

Репозиторій: https://github.com/mykhailokrainiak/web-progr
Папка з лабораторною роботою: https://github.com/mykhailokrainiak/web-progr/tree/main/lab

Додаток. Вивід Newman (results/newman-cli.txt)

```
newman

ЛР 10 — HTTP і Postman (Крайняк М. Ю., 451)

□ 01 — Основний сценарій
└ 01 — Список завдань
  GET http://127.0.0.1:8080/api.php?resource=tasks [200 OK, 545B, 41ms]
  √  Статус 200 OK
  √  Відповідь JSON
  √  data — масив, meta.count = довжина
  √  Записів: 2
  √  Початкові записи id 1 і 2

└ 02 — Фільтр виконаних (done=1)
  GET http://127.0.0.1:8080/api.php?resource=tasks&done=1 [200 OK, 428B, 10ms]
  √  Статус 200 OK
  √  Відповідь JSON
  √  data — масив, meta.count = довжина
  √  Записів: 1
  √  Лише done = true
  √  Це завдання id 2

└ 02a — Фільтр невиконаних (done=0)
  GET http://127.0.0.1:8080/api.php?resource=tasks&done=0 [200 OK, 407B, 8ms]
  √  Статус 200 OK
  √  Відповідь JSON
  √  data — масив, meta.count = довжина
  √  Записів: 1
  √  Лише done = false

└ 02b — Один запис (id=1)
  GET http://127.0.0.1:8080/api.php?resource=tasks&id=1 [200 OK, 335B, 13ms]
  √  Статус 200 OK
  √  Відповідь JSON
  √  data — об’єкт id 1
  √  Для одного запису немає meta

└ 02c — id разом із done
  GET http://127.0.0.1:8080/api.php?resource=tasks&id=1&done=1 [400 Bad Request, 358B, 9ms]
  √  Статус 400 Bad Request
  √  Відповідь JSON
  √  error.message пояснює причину
  √  Немає data

└ 03 — URL-encoded форма
  POST http://127.0.0.1:8080/api.php?resource=echo [200 OK, 462B, 48ms]
  √  Статус 200 OK
  √  Відповідь JSON
  √  content_type — urlencoded
  √  Поля тіла в $_POST
  √  resource — з URL, а не з тіла
  √  Файла немає

└ 04 — Створити завдання
  POST http://127.0.0.1:8080/api.php?resource=tasks [201 Created, 380B, 17ms]
  √  Статус 201 Created
  √  Відповідь JSON
  √  Новий запис
  √  Location веде на новий запис

└ 05 — Прочитати створене
  GET http://127.0.0.1:8080/api.php?resource=tasks&id=3 [200 OK, 336B, 8ms]
  √  Статус 200 OK
  √  Той самий запис, що створив POST
```

```
└ 06 — Оновити завдання (PUT)
  PUT http://127.0.0.1:8080/api.php?resource=tasks&id=3 [200 OK, 334B, 21ms]
  √  Статус 200 OK
  √  Відповідь JSON
  √  id незмінний, нові title і done

└ 06a — PUT без done
  PUT http://127.0.0.1:8080/api.php?resource=tasks&id=3 [422 Unknown Status Code, 399B, 8ms]
  √  Статус 422 Unprocessable Content
  √  Відповідь JSON
  √  error.message пояснює причину
  √  Немає data

└ 07 — Підтвердити збереження PUT
  GET http://127.0.0.1:8080/api.php?resource=tasks&id=3 [200 OK, 334B, 7ms]
  √  Статус 200 OK
  √  Збережено PUT, а не невдалий PUT без done

└ 08 — Видалити завдання
  DELETE http://127.0.0.1:8080/api.php?resource=tasks&id=3 [204 No Content, 229B, 10ms]
  √  Статус 204 No Content
  √  Тіло порожнє
  √  Немає Content-Type JSON

└ 09 — Перевірити відсутність
  GET http://127.0.0.1:8080/api.php?resource=tasks&id=3 [404 Not Found, 320B, 7ms]
  √  Статус 404 Not Found
  √  Відповідь JSON
  √  error.message пояснює причину
  √  Немає data

└ 09a — Повторний DELETE
  DELETE http://127.0.0.1:8080/api.php?resource=tasks&id=3 [404 Not Found, 320B, 10ms]
  √  Статус 404 Not Found
  √  Відповідь JSON
  √  error.message пояснює причину
  √  Немає data

└ 10 — Список після циклу CRUD
  GET http://127.0.0.1:8080/api.php?resource=tasks [200 OK, 545B, 6ms]
  √  Статус 200 OK
  √  Відповідь JSON
  √  data — масив, meta.count = довжина
  √  Записів: 2
  √  Видаленого запису немає, інші залишилися

□ 02 — Помилкові запити
└ 11 — Порожній title
  POST http://127.0.0.1:8080/api.php?resource=tasks [422 Unknown Status Code, 378B, 37ms]
  √  Статус 422 Unprocessable Content
  √  Відповідь JSON
  √  error.message пояснює причину
  √  Немає data

└ 12 — Неправильні типи
  POST http://127.0.0.1:8080/api.php?resource=tasks [422 Unknown Status Code, 378B, 7ms]
  √  Статус 422 Unprocessable Content
  √  Відповідь JSON
  √  error.message пояснює причину
  √  Немає data

└ 12a — Рядок "false" замість false
  POST http://127.0.0.1:8080/api.php?resource=tasks [422 Unknown Status Code, 399B, 10ms]
  √  Статус 422 Unprocessable Content
  √  Відповідь JSON
  √  error.message пояснює причину
  √  Немає data

└ 12b — Поле id у тілі
  POST http://127.0.0.1:8080/api.php?resource=tasks [422 Unknown Status Code, 376B, 9ms]
```

```
  √  Статус 422 Unprocessable Content
  √  Відповідь JSON
  √  error.message пояснює причину
  √  Немає data

└ 13 — Пошкоджений JSON
  POST http://127.0.0.1:8080/api.php?resource=tasks [400 Bad Request, 324B, 30ms]
  √  Статус 400 Bad Request
  √  Відповідь JSON
  √  error.message пояснює причину
  √  Немає data

└ 14 — Невідповідний Content-Type
  POST http://127.0.0.1:8080/api.php?resource=tasks [415 Unsupported Media Type, 374B, 6ms]
  √  Статус 415 Unsupported Media Type
  √  Відповідь JSON
  √  error.message пояснює причину
  √  Немає data

└ 15 — Масив у query (id[])
  GET http://127.0.0.1:8080/api.php?resource=tasks&id[]=1 [400 Bad Request, 332B, 8ms]
  √  Статус 400 Bad Request
  √  Відповідь JSON
  √  error.message пояснює причину
  √  Немає data

└ 16 — Невідомий запис
  GET http://127.0.0.1:8080/api.php?resource=tasks&id=999999999 [404 Not Found, 320B, 6ms]
  √  Статус 404 Not Found
  √  Відповідь JSON
  √  error.message пояснює причину
  √  Немає data

└ 17 — Невідомий ресурс
  GET http://127.0.0.1:8080/api.php?resource=unknown [404 Not Found, 316B, 7ms]
  √  Статус 404 Not Found
  √  Відповідь JSON
  √  error.message пояснює причину
  √  Немає data

└ 18 — Непідтримуваний метод PATCH
  PATCH http://127.0.0.1:8080/api.php?resource=tasks [405 Method Not Allowed, 397B, 8ms]
  √  Статус 405 Method Not Allowed
  √  Відповідь JSON
  √  error.message пояснює причину
  √  Немає data
  √  Allow перелічує підтримувані методи

└ 19 — DELETE без id
  DELETE http://127.0.0.1:8080/api.php?resource=tasks [400 Bad Request, 324B, 7ms]
  √  Статус 400 Bad Request
  √  Відповідь JSON
  √  error.message пояснює причину
  √  Немає data

└ 20 — Невдалі POST не створили записів
  GET http://127.0.0.1:8080/api.php?resource=tasks [200 OK, 545B, 12ms]
  √  Статус 200 OK
  √  Відповідь JSON
  √  data — масив, meta.count = довжина
  √  Записів: 2
  √  Лише початкові id 1 і 2

□ 03 — Basic Auth
└ 21 — Без автентифікації
  GET http://127.0.0.1:8080/api.php?resource=auth [401 Unauthorized, 406B, 7ms]
  √  Статус 401 Unauthorized
  √  Відповідь JSON
  √  error.message пояснює причину
  √  Немає data
  √  WWW-Authenticate: Basic
```

```
└ 22 — Basic Auth (демонстраційні дані)
  GET http://127.0.0.1:8080/api.php?resource=auth [200 OK, 308B, 10ms]
  √  Статус 200 OK
  √  authenticated = true
  √  Cookie входу не створено

└ 23 — Basic Auth з неправильним паролем
  GET http://127.0.0.1:8080/api.php?resource=auth [401 Unauthorized, 406B, 6ms]
  √  Статус 401 Unauthorized
  √  Відповідь JSON
  √  error.message пояснює причину
  √  Немає data

□ 04 — Пошук q (власне доповнення)
└ 24 — q великими літерами
  GET http://127.0.0.1:8080/api.php?resource=tasks&q=ПЕРЕВІРИТИ [200 OK, 407B, 8ms]
  √  Статус 200 OK
  √  Відповідь JSON
  √  data — масив, meta.count = довжина
  √  Записів: 1
  √  Знайдено «Перевірити GET» без урахування регістру

└ 25 — q іншим регістром у середині слова
  GET http://127.0.0.1:8080/api.php?resource=tasks&q=ТестОві [200 OK, 428B, 7ms]
  √  Статус 200 OK
  √  Відповідь JSON
  √  data — масив, meta.count = довжина
  √  Записів: 1
  √  Знайдено id 2

└ 26 — q без збігів
  GET http://127.0.0.1:8080/api.php?resource=tasks&q=автобус [200 OK, 286B, 7ms]
  √  Статус 200 OK
  √  Відповідь JSON
  √  data — масив, meta.count = довжина
  √  Записів: 0
  √  Порожній масив, не помилка

└ 27 — q разом із done=0
  GET http://127.0.0.1:8080/api.php?resource=tasks&q=и&done=0 [200 OK, 407B, 7ms]
  √  Статус 200 OK
  √  Відповідь JSON
  √  data — масив, meta.count = довжина
  √  Записів: 1
  √  Обидва фільтри: лише невиконане id 1

└ 27a — Той самий q без done
  GET http://127.0.0.1:8080/api.php?resource=tasks&q=и [200 OK, 545B, 7ms]
  √  Статус 200 OK
  √  Відповідь JSON
  √  data — масив, meta.count = довжина
  √  Записів: 2

└ 28 — Масив q[]
  GET http://127.0.0.1:8080/api.php?resource=tasks&q[]=Перевірити [400 Bad Request, 331B, 7ms]
  √  Статус 400 Bad Request
  √  Відповідь JSON
  √  error.message пояснює причину
  √  Немає data

└ 29 — q разом з id
  GET http://127.0.0.1:8080/api.php?resource=tasks&id=1&q=Перевірити [400 Bad Request, 355B, 7ms]
  √  Статус 400 Bad Request
  √  Відповідь JSON
  √  error.message пояснює причину
  √  Немає data

└ 30 — q довший за 40 символів
  GET http://127.0.0.1:8080/api.php?resource=tasks&q=aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa [400 Bad
Request, 363B, 21ms]
```

```
  √  Статус 400 Bad Request
  √  Відповідь JSON
  √  error.message пояснює причину
  √  Немає data

└ 31 — q лише з пробілів = без пошуку
  GET http://127.0.0.1:8080/api.php?resource=tasks&q=    [200 OK, 545B, 7ms]
  √  Статус 200 OK
  √  Відповідь JSON
  √  data — масив, meta.count = довжина
  √  Записів: 2

└ 32 — q з пробілами по краях (trim)
  GET http://127.0.0.1:8080/api.php?resource=tasks&q=  дані   [200 OK, 428B, 9ms]
  √  Статус 200 OK
  √  Відповідь JSON
  √  data — масив, meta.count = довжина
  √  Записів: 1
  √  Знайдено id 2

□ 05 — Ручне завантаження файла
└ Вручну — Вкладення attachment
  POST http://127.0.0.1:8080/api.php?resource=echo
[200 OK, 528B, 119ms]
  √  Статус 200 OK
  √  multipart/form-data
  √  Текстове поле name
  √  Метадані файла
```

	executed	failed
iterations	1	0
requests	41	0
test-scripts	41	0
prerequest-scripts	1	0
assertions	172	0
total run duration: 4.1s		
total data received: 6.17kB (approx)		
average response time: 14ms [min: 6ms, max: 119ms, s.d.: 19ms]		